

Monolith vs Microservices

- A monolith architecture means an entire application is built and deployed as one single unit.
- Everything runs together :
 - Authentication → APIs
 - All inside one codebase → Payments → Notifications.
 - Orders
 - & one deployment.

Example :- E-commerce app :- one backend app handles :-

- User Login → Orders → Payments
- Product catalog → Admin panel.

Single server (or replicated copies of same app)

Why monoliths are used?

- Simpler to build
- Easier to deploy
- Easier to debug
- Easier to test

• Better for small teams.

(for startups, it is often the best choice)

Problems at scale

- As system grows :

- codebase becomes large
- Deployments become risky
- Scaling specific features become hard
- One bug can crash entire system
- one module affects entire application.

- Microservices architecture means the application is split into multiple independent services.
- Each services has its own responsibility and can be deployed independently, often has its own database.

Example :-

- Auth service
- Order service
- Payment service
- Notification service

- They communicate over network (HTTP, message queues)

Why Microservices?

- Independent scaling
- Independent deployments
- Easier team ownership
- Better fault isolation
- Technology flexibility.

Ex!:- If payments are heavy load, scale only payment services.

Problems with Microservices

- They introduce:-
 - Network complexity
 - Distributed failure
 - Monitoring challenges
 - Higher DevOps ~~overhead~~ overhead
 - Microservices when large team, high traffic, independent scaling required, multiple business domains.
 - Monolith when small team, early stage, simple product, low traffic.
- * Do not start with microservices unless necessary.

Monolith to Microservices Migration

* Monolith is good for early stage but as system grows:-
at some point we should split this into microservices.

→ But migration is risky. - If done wrong:-

- System breaks
- Data inconsistencies
- Performance drops.

When you should migrate?

→ Do not follow microservices trend. Migrate only when:

- Team needs independent deployment
- Codebase is hard to manage
- Performance bottlenecks are isolated.
- Modules scale differently.

Migration Strategy

1) Identify clear boundaries. Split system by business domains.

2) Extract a service :- 'Notification Service' (Ex)

New flow :- Monolith → Calls Notification Service via API

Keep rest inside monolith.

3) DB Strategy :- Initially, service may share monolith DB.

But long term :- Each service should own its database.

The DB separation should be gradual.

4) Introduce API Gateway :- As services increase :-

- Add :-
- API Gateway
 - Central routing
 - Authentication layer.